ConferenceCheck Mobile: Analytics, Meals and Notifications: Validation Report

Project: ConferenceCheck Mobile: Analytics, Meals and Notifications (component 4.3b) **Student:** Ndashi Bwalya Chisanga, computer number 2021470105 **Supervisor:** Mr. Mofya Phiri **Repository:** https://github.com/ndashi-chisanga6/ConferenceCheck-Mobile-Analytics-Meals-and-Notifications **Validated state:** tag `resubmission-16-october`

This document reports the validation results and describes the path that produces them: the seeded dataset, the commands that reproduce every figure, and the acceptance criteria each run is held to (proposal Section 5, Table 3). Every number below is taken from a committed results file or from a named test or command output, and the source is named alongside it.

This version answers the supervisor's comments on the version at tag `final-resubmission` (submission 145): it adds the end-to-end voucher trial on a device (Section 11), a test-case table (Section 12), precise wording for the freshness and export results (Sections 0, 4 and 5), and the frontend build step in the commands (Section 10.2). It is measured against the approved proposal, v4 (submission 144).

The version at `final-resubmission` in turn answered the comments on the one at tag `milestone-12-final-submission`: the per-criterion status replaces the earlier claim that every criterion was met, the concurrency result now comes from an experiment that can tell the design apart from a naive one, export correctness is checked against SQL rather than against another API endpoint, dashboard freshness is measured rather than derived, and access control and offline replay are covered (Sections 7 and 8). The work behind these changes is listed with its commits in `docs/remaining_work.md`.

Conventions. Latencies are quoted in milliseconds to one decimal place. Counts and rejection rates are exact. All timings are wall-clock, from named runs on a local PHP 8.5.8 / PostgreSQL 18.2 stack on 10 October 2026.

Contents

Section	Subject	Page
0	Headline results	2
0.1	Auditable evidence	4
1	Meal voucher redemption correctness	6
2	Session attendance and capacity correctness	7
3	Latency against the proposal's targets	8

Section	Subject	Page
4	Analytics dashboard freshness	9
5	Export correctness	9
6	Notification delivery	10
7	Access control	11
8	Offline replay	12
9	Baseline comparison, and its limits	13
10	Reproducibility	14
10.1	The environment these commands assume	14
10.2	Commands	14
11	End-to-end voucher trial on a device	15
12	Test cases	16

0. Headline results

The evaluation exercise seeds a 500-attendee event with three meal categories, a voucher issued per attendee per category for 1,500 vouchers in all, and ten sessions (`tools/seed-evaluation-event.php`), then drives it through the real API rather than through unit-level mocks.

Criterion (proposal Table 3)	Target	Result	Status
Duplicate redemption rejection, including concurrent submissions	100%	100/100 sequential re-scans refused; on PostgreSQL behind five server processes, 50/50 five-way races with exactly one redemption with the lock, the constraint or both in place; with both removed, 48/50 vouchers redeemed more than once (Section 1)	Met
Scan validation latency	median < 500 ms, p95 < 1 s	voucher scan 145.3 ms / 261.0 ms (n=200); session scan 128.4 ms / 228.7 ms (n=40)	Met on loopback only (Section 3)
Dashboard freshness	≤ 30 s behind the database	20 injected check-ins, measured with a poller that reproduces the app's refresh, not the app on a device: median 18.1 s, p95 28.4 s, max 28.5 s	20 of 20 within 30 s in the recorded run; not guaranteed by design (a rerun gave 19 of 20, one at

Criterion (proposal Table 3)	Target	Result	Status
			30.02 s) (Section 4)
Capacity warning before 100% occupancy	warning delivered before full	two automated transition tests; one observation on an emulator	Partly met: shown, not timed
Push delivery	≥ 95% of online devices within 30 s	one device, one observed delivery of about 5 s	Not measured
In-app inbox reach	100% of recipients on next open	a recipient record for every recipient, inbox reads filtered to recipients and tested; not measured on devices	Partly met
Export correctness	row counts and totals equal to the database	all four exports equal to direct SQL on row counts and on a total each; `notifications.csv` checked as 0 against 0, its real check being a unit test; the sessions total has one non-zero session of ten (Section 5)	Met, with those two limits
Comparison with a manual baseline	measured against a staffed paper line	not run (Section 9)	Not run

Two criteria are met, one of them on loopback only, and export correctness is met with the two limits stated in its row; dashboard freshness held in the recorded run but is not guaranteed by design; two criteria are partly met; push delivery across a fleet was not measured; and the manual baseline was not run. The voucher flow also works end to end in the app, in one trial on an emulator with manual token entry (Section 11). Two further results test the guarantees the design rests on: all three categories were redeemable independently for 20 of 20 attendees, and 20 of 20 sequential duplicate session scans were refused.

Sources: `tools/evaluation-results.json` (`python tools/run-evaluation.py` against `tools/seed-evaluation-event.php`), `tools/concurrency-results.json` (`python tools/run-concurrency-experiment.py`) and `tools/freshness-results.json` (`python tools/measure-dashboard-freshness.py`).

0.1 Auditable evidence

Validated state. Tag `final-resubmission`. The exact commit it resolves to is verifiable with:

```
git rev-list -n1 final-resubmission
```

(`final-resubmission` is an annotated tag; a bare `git rev-parse` on it returns the tag object's own hash, not the commit, so use `rev-list -n1` or `rev-parse final-resubmission^{commit}` to resolve to the commit.)

Test suite. Command and output, run on PHP 8.5.8 with PostgreSQL 18.2 on 10 October 2026:

```
$ php artisan test
{"tool":"phpunit","result":"passed","tests":70,"passed":70,"assertions":288}
```

Of those 70 tests, 31 (152 assertions) are this component's, all in `tests/Feature/ConferenceApiTest.php`. The other 39 are the Laravel starter kit's example, dashboard, authentication and settings tests, which came with the application scaffold and test none of this component's behaviour. Twelve of the 39 render pages and need the frontend built (`npm run build`) before they pass. The 31 component tests are:

- authentication and roles: `test_login_returns_token`, `test_register_cannot_pick_organiser_role`
- analytics: `test_analytics_summary_endpoint`
- meal vouchers: `test_valid_meal_voucher_scan`, `test_duplicate_meal_voucher_scan_rejected`, `test_constraint_rejection_returns_409_not_500`, `test_invalid_meal_voucher_scan_rejected`, `test_redeeming_one_category_leaves_the_attendees_other_categories_redeemable`, `test_redemptions_cannot_be_edited_or_deleted_away`
- sessions and check-in: `test_valid_session_attendance_scan`, `test_duplicate_session_attendance_scan_rejected`, `test_session_scan_that_loses_a_race_gets_409_not_500`, `test_check_in_that_loses_a_race_is_a_duplicate_not_a_500`, `test_session_scan_records_the_scanning_device`, `test_capacity_threshold_transitions_alert_organisers`
- notifications: `test_notification_send_in_demo_mode`, `test_missing_firebase_config_fails_instead_of_faking_success`, `test_custom_target_is_rejected_instead_of_sent_to_everyone`, `test_pushes_go_out_after_the_recipient_rows_are_committed`, `test_notification_send_uses_fcm_v1_when_configured`, `test_delivery_is_recorded_against_each_recipient_individually`
- reports: `test_csv_report_endpoint_returns_downloadable_csv`, `test_every_export_has_one_row_per_database_record`
- access control: `test_attendee_can_fetch_own_qr_token`, `test_my_attendee_returns_404_when_no_record_is_linked`,

```
test_attendee_cannot_read_staff_or_organiser_routes,
test_attendee_can_only_open_their_own_voucher,
test_notifications_are_only_readable_by_their_recipients,
test_only_the_owner_can_delete_a_device_token,
test_linking_an_attendee_record_does_not_demote_staff,
test_scanner_cannot_download_reports
```

The suite was rerun on 14 October 2026, after `npm run build`, with the same result: 70 passed, 288 assertions.

PHPStan level 7: clean. Pint: clean. Flutter: `flutter analyze` no issues, `flutter test` 30/30 passed (27 before 14 October, plus three for the My meals screen in `mobile/test/my_meals_screen_test.dart`).

Re-verification, 10 October 2026. Every command in Section 10 was run on that date against the code at the validated state. The evaluation seed produced 500 attendees, 3 meal categories, 1,500 vouchers and 10 sessions; the evaluation round reported 200 of 200 scans accepted, 100 of 100 sequential duplicates refused, 20 of 20 single-server races with exactly one success, 20 of 20 attendees redeeming all three categories independently, 20 of 20 sequential duplicate session scans refused, and all four exports equal to direct SQL. The concurrency experiment and the freshness measurement were run the same day; their results are in Sections 1 and 4.

Route coverage, and which routes are this component's. The backend exposes 47 API routes (`php artisan route:list --path=api`), distributed as: `MealController` 10, `AttendeeController` 8, `SessionController` 7, `NotificationController` 5, `EventController` 5, `AnalyticsController` 4, `ReportController` 4, `AuthController` 4.

The repository is a standalone Laravel 13 application, started from the Laravel starter kit, that implements the API contract agreed with component 4.3a. It is not a shared backend, and component 4.3a's own API is not part of it. Two of the eight `AttendeeController` routes, `POST /events/{event}/attendees/check-in/scan` and `POST /events/{event}/attendees/{attendee}/check-in`, are a minimal check-in stub written for this component, as agreed, so that it can be built and tested on its own. Check-in capture is component 4.3a's work; this component only reads the check-in data, for analytics. Integration with component 4.3a's own API was not demonstrated. `AuthController` and `EventController` are the authentication and event scaffolding both components need. The routes this component contributes are the meal voucher, session attendance, notification, analytics and reporting groups, plus `GET /events/{event}/attendees/me`, which lets the attendee QR screen fetch a real server-issued token.

The boundary is visible in the mobile codebase, which is this component's alone: `mobile/lib/features/` contains `dashboard`, `meals`, `sessions`, `notifications`, `reports`, `attendee`, `events`, `auth`, `profile` and `sync`, and no check-in capture module. The only mobile code that mentions check-in at all is in `features/dashboard/`, where check-in counts are read back from the analytics endpoints.

1. Meal voucher redemption correctness

How a scan is decided. `MealController::scanVoucher` (`app/Http/Controllers/Api/MealController.php`) runs the redemption in one database transaction. It selects the voucher by its token with `lockForUpdate()`, so on PostgreSQL a second scan of the same voucher waits for the first transaction to finish. It then checks, in order, that the voucher belongs to the event (404 otherwise), that its status is `unused` (409 otherwise, with the status and the time of the earlier redemption), that the category is active and that the scan is inside the redemption window (422 otherwise). Only then does it mark the voucher redeemed and insert the `meal_redemptions` row, whose `meal_voucher_id` is unique. If that insert violates the unique constraint, the violation is caught outside the transaction and returned as the same 409. With the lock in place, a losing scan is refused by the status check after it has waited for the winner; the unique constraint is the backstop if the lock is missing.

Sequential re-scans: 100 of 100 refused after the first successful redemption.

Concurrent races, single server. In `tools/run-evaluation.py`, 20 of 20 races of five simultaneous scans resolved to exactly one success. The development server used for that run executes one request at a time, so this result cannot tell the design apart from a plain check-then-insert, and it is not the evidence for the concurrency claim. That evidence is the experiment below.

Concurrent races on PostgreSQL behind five server processes. `tools/run-concurrency-experiment.py` runs five separate PHP server processes against PostgreSQL and fires each race's five scans of one fresh voucher at the same instant, one to each process, so the requests overlap in the database. It runs every combination of the two safeguards, row lock on or off and unique constraint on or off, with the status check present in all of them. Each configuration is served from a scratch git worktree of the same commit, with `lockForUpdate()` deleted where the lock is off, and the constraint is dropped on a scratch database. The grid is run twice: as the code stands, and with a 50 ms pause injected between the status check and the write in every configuration, which holds the window in which two scans can both pass the check open on purpose. Fifty races per configuration:

Configuration	Races with exactly one success	Vouchers redeemed more than once	Responses
lock on, constraint on (as built)	50/50	0	50 × 200, 200 × 409
lock off, constraint on	50/50	0	50 × 200, 200 × 409
lock on, constraint off	50/50	0	50 × 200, 200 × 409
lock off, constraint off	2/50	48	190 × 200, 60 × 409

Configuration	Races with exactly one success	Vouchers redeemed more than once	Responses
lock on, constraint on, 50 ms window	50/50	0	50 × 200, 200 × 409
lock off, constraint on, 50 ms window	50/50	0	50 × 200, 200 × 409
lock on, constraint off, 50 ms window	50/50	0	50 × 200, 200 × 409
lock off, constraint off, 50 ms window	0/50	50	236 × 200, 14 × 409

The naive configuration fails even without the injected pause, so the race is real under this load, and the test is able to detect it. With either safeguard in place there was exactly one redemption in every race, and no response was a 500. Each safeguard is therefore sufficient on its own, and the system as built carries both. What this does not measure is throughput: it tests correctness under contention, not how many scanners the system can serve at once.

Source: `tools/concurrency-results.json`.

Cross-category independence: 20 of 20 reserved attendees, each holding a voucher in all three categories, redeemed every category successfully and were then refused on a repeat of the first. Redeeming breakfast consumes breakfast and nothing else. The same guarantee is pinned at unit level by `test_redeeming_one_category_leaves_the_attendees_other_categories_redeemable`.

Ground truth: 280 redemption rows for 200 batch scans, 20 race winners and 60 cross-category redemptions, equal to a direct SQL count (Section 5).

The redemption record. A redemption cannot be changed or removed once it is recorded: the model refuses updates and deletes, the foreign keys from `meal_redemptions` restrict deletes rather than cascading them, and the API answers 409 instead of deleting an event, attendee or meal category that has redemptions. This is pinned by `test_redemptions_cannot_be_edited_or_deleted_away`.

Source: `sequential_duplicate_rejection`, `concurrent_duplicate_rejection` and `cross_category_independence` in `tools/evaluation-results.json`.

2. Session attendance and capacity correctness

The unique `(session, attendee)` constraint prevents double counting; 20 of 20 sequential duplicate session check-ins were refused in the evaluation run.

`SessionController::scan` locks the session row inside its transaction, which serialises scans of one session, so the duplicate check, the capacity count and the capacity transition are all

computed under the lock. A scan that still reaches the unique constraint returns 409, not 500. That path is tested by `test_session_scan_that_loses_a_race_gets_409_not_500`, which inserts a competing row at the moment between the check and the insert. The session path was not part of the parallel experiment in Section 1, so it has not been tested under real parallel load.

Capacity threshold transitions (the configurable warning fraction, default 90%, and over-capacity) push an automatic organiser notification, sent after the attendance transaction commits. This is covered by `test_capacity_threshold_transitions_alert_organisers` and was observed once on an emulator (`docs/evidence-capacity-alerts.png`). It was not timed, so the criterion that the warning arrives before the session is full is shown, not measured.

Source: `session_duplicate_rejection` in `tools/evaluation-results.json`.

3. Latency against the proposal's targets

Nielsen's response-time thresholds, cited in the proposal's literature review, set the sub-second budget for scan-validation round trips at a catering service point where queue throughput matters.

Endpoint	n	Mean	Median	p95	Max	Target (median / p95)
Meal voucher scan	200	164.0 ms	145.3 ms	261.0 ms	316.6 ms	< 500 ms / < 1,000 ms
Session scan	40	143.3 ms	128.4 ms	228.7 ms	242.1 ms	< 500 ms / < 1,000 ms
Analytics summary	20	124.0 ms	117.9 ms	138.7 ms	208.2 ms	part of each dashboard refresh (Section 4)

These were measured over loopback on one development machine. The proposal's targets were written for venue Wi-Fi, which this run does not exercise.

The envelope these numbers hold in. These figures are one named run and a property of that run rather than of the system. Four runs of the evaluation script have been recorded on this machine: voucher-scan medians of 114.0 ms, 199.0 ms, 187.5 ms and 145.3 ms, with 95th percentiles of 210.5 ms, 304.3 ms, 286.7 ms and 261.0 ms. The spread is machine load, not code. On a development machine the voucher-scan median falls between roughly 110 ms and 200 ms and the 95th percentile between roughly 210 ms and 305 ms, inside the 500 ms and 1,000 ms targets in every run observed. Behaviour on venue Wi-Fi, behind a production server, or above 500 attendees is not established.

Source: `voucher_scan_latency`, `session_scan_latency` and `analytics_summary_latency` in `tools/evaluation-results.json`.

4. Analytics dashboard freshness

The dashboard polls rather than streams (a documented trade-off against a WebSocket layer, proposal Section 8). The app fetches the four analytics endpoints in parallel and schedules its next refresh 30 seconds after the previous one started (`mobile/lib/features/dashboard/application/analytics_providers.dart`).

Freshness was measured the way the proposal describes, by injecting check-ins and timing their appearance. `tools/measure-dashboard-freshness.py` runs a poller that reproduces the app's refresh exactly, injects 20 check-ins one at a time at random moments through the check-in API, and times each from the start of the injecting request to the end of the first dashboard fetch that shows it.

Run	Trials	Median	p95	Max	Within 30 s
Recorded run, 10 October 2026	20	18.1 s	28.4 s	28.5 s	20/20
Supervisor's rerun of the same script	20	11.4 s	not reported	30.02 s	19/20

The result is 20 of 20 within 30 s in the recorded run, not guaranteed by design. The measurement is a poller that reproduces the app's refresh, not the app on a device: it covers the data path, polling plus API, and not the time Flutter takes to repaint. By design the worst case is one full 30 s cycle plus the duration of a fetch, so a check-in that lands just after a fetch starts can appear fractionally later than 30 s. The supervisor's rerun hit exactly that case once, at 30.02 s.

Source: `tools/freshness-results.json`.

5. Export correctness

Each of the four CSV exports was downloaded through the API and compared with SQL run directly on the database, on its row count and on one total per export.

Export	Check	CSV	SQL	Match
`attendance.csv`	rows	500	500	yes
`attendance.csv`	attendees checked in	40	40	yes
`meals.csv`	rows	280	280	yes
`meals.csv`	redemptions per category	94 / 93 / 93	94 / 93 / 93	yes
`sessions.csv`	rows	10	10	yes
`sessions.csv`	attendance per	40, then 0 for the other nine (one	same	yes

Export	Check	CSV	SQL	Match
	session	non-zero session of ten)		
notifications.csv	rows	0	0	yes
notifications.csv	total recipients	0	0	yes

The evaluation event has no notifications, so `notifications.csv` was checked as 0 against 0 in the run, which shows only that the export is well formed. Its real check is the unit test named below. The sessions total is also thin: only one of the ten sessions has any attendance, so it compares one non-zero value and nine zeros. A notification was not sent during the run because the local configuration uses live Firebase credentials. That export's row count is tested on seeded data, which does contain a notification, by `test_every_export_has_one_row_per_database_record`.

The exports read their rows in chunks of 1,000 while the file is being written (`lazy()` in `app/Http/Controllers/Api/ReportController.php`), so the whole export is not held in memory at once.

Source: `export_correctness` in `tools/evaluation-results.json`.

6. Notification delivery

Delivery is recorded per recipient. A recipient is `sent` only when one of that recipient's own device tokens was accepted, `failed` with a reason when all of theirs were refused, and left `pending` when they have no registered device and will receive the message through the in-app inbox instead. Before this was fixed, one aggregate outcome was applied to every recipient. The behaviour is pinned by `test_delivery_is_recorded_against_each_recipient_individually`, which fails against the earlier implementation. Recipient rows are committed before anything is pushed, and the Firebase calls run outside the database transaction (`test_pushes_go_out_after_the_recipient_rows_are_committed`).

Demo mode is never counted as delivery. Firebase demo mode is off unless it is set explicitly. When it is on, nothing is pushed, a warning is logged, the response carries `"demo": true`, and the notification and every recipient are recorded as `demo`, never `sent`. A missing Firebase configuration with demo mode off is a failed send, not a silent demo (`test_missing_firebase_config_fails_instead_of_faking_success`).

Live delivery, as single observations. Live Firebase Cloud Messaging v1 delivery was observed end to end once on an Android emulator: an organiser send reached the device's notification shade (`docs/evidence-push-delivered.png`). Receipt took about 5 seconds on that one device. No run log of fleet delivery was committed, so the proposal's target of 95% of online devices within 30 seconds is not measured.

Inbox. Every notification is stored with a recipient record for each person it was sent to, and the in-app inbox shows a user only the notifications they were a recipient of (Section 7). This is tested; it was not measured on devices.

7. Access control

The supervisor found that an authenticated attendee assigned to the event could read another attendee's voucher token, the attendance export, the event's redemption history and a notification addressed to organisers, on a local database with synthetic data. Each was fixed and has a feature test that expects 403 and that fails against the earlier code:

Defect	Fix	Test
Attendee could open any voucher by id	an attendee can open only their own voucher; organisers and scanners can open any	`test_attendee_can_only_open_their_own_voucher`
Attendee could download the CSV reports and read the redemption history, the attendee list, the voucher list and session attendance	reports are organiser only; the other four are staff (organiser and scanner) only	`test_attendee_cannot_read_staff_or_organiser_routes`, `test_scanner_cannot_download_reports`
Attendee could list every notification and open	the list holds only notifications the user received,	`test_notifications_are_only_readable_by_their_recipients`

Defect	Fix	Test
organiser-only ones	and opening any other returns 403; only organisers see recipient rows	

Scanners keep read access to the redemption history, because the scanner app's history tab uses it.

Four further defects of the same kind were found while fixing these and are covered the same way: registration let a caller choose the organiser role (`test_register_cannot_pick_organiser_role`); any organiser could delete any user's device token (`test_only_the_owner_can_delete_a_device_token`); the `custom` notification target passed validation and was sent to every attendee (`test_custom_target_is_rejected_instead_of_sent_to_everyone`); and linking an attendee record to a scanner demoted them to attendee (`test_linking_an_attendee_record_does_not_demote_staff`).

8. Offline replay

Scans made without connectivity are queued on the device and replayed later (`mobile/lib/features/sync/application/scan_sync_controller.dart`). Two defects in the replay path were fixed:

- **Recoverable failures were dropped.** Replay treated every HTTP error as a settled result, so an expired login or a server fault silently lost the scan. Now only a business answer settles a queued scan: 404 (invalid token), 409 (already redeemed or duplicate) and 422 (not redeemable now). Anything else, including no connection, 401, 403, 429 and 5xx, keeps that scan and every scan after it, and stops the run.
- **A concurrent enqueue could be overwritten.** Replay wrote the leftover list back over the whole queue, so a scan queued while a replay was running was wiped. Replay now removes only the scans it handled.

Tests in `mobile/test/scan_replay_test.dart` and `mobile/test/scan_replay_restart_test.dart` drive the replay itself with a fake API:

Test	What it shows
only business answers settle a replay	404, 409 and 422 settle; no connection, 401, 403, 408, 429, 500, 502 and 503 do not

Test	What it shows
an expired login keeps the scan and everything after it	a 401 mid-queue keeps that scan and the rest
a server fault keeps the scan for the next sync	a 500 keeps the queue
a scan queued while a replay is running is not lost	a scan added mid-replay survives
a scan queued during a failing replay stays behind the kept ones	queue order is preserved
already redeemed and invalid tokens are dropped	business answers clear the queue
scans queued before a restart are replayed after it	a queue left by an earlier app run, meal and session scans, is replayed
a scan kept after an auth failure survives a restart and syncs later	a 401 survives a restart and syncs on the next run

These run against a fake API in the test harness; replay was not exercised on a device.

Service rule for meal scans made offline. A queued meal scan is not a redemption. The server has not checked the voucher, so the meal is not served until the server answers 200. The scanner now says so when it queues a meal scan: "No connection — voucher NOT redeemed. Do not serve this meal; the scan is saved and will be checked when back online." (`meal_scan_controller.dart`). The earlier message, "scan saved and will sync", read as though the meal could be served. Camera decoding of QR codes was checked by hand and not measured: `mobile/test/qr_parser_test.dart` tests only the trimming of a manually entered token.

9. Baseline comparison, and its limits

The proposal's evaluation plan calls for comparing the voucher module against a simulated, human-staffed paper-register meal line. That baseline was **not** conducted: organising a staffed comparison exercise was not feasible within the project's scope. This is stated plainly rather than approximated, because a fabricated baseline number would be worse than admitting the gap.

What the evaluation does establish is the system-side bound: a median scan-to-decision time of 145.3 ms, with single-use enforcement that a paper register cannot provide at any speed (a photographed or reused paper voucher has no server-side check to fail).

10. Reproducibility

10.1 The environment these commands assume

PHP 8.5.8 for Windows, with the `pdo_pgsql` and `pgsql` extensions enabled in its `php.ini`, along with `openssl`, `curl`, `mbstring`, `zip`, `gd` and `fileinfo`. The PostgreSQL client library `libpq.dll` ships beside the executable. A PHP without those extensions loaded cannot connect to the database and none of the commands below will run.

The repository carries `php-pgsql.bat`, a one-line wrapper that resolves to a PHP under a `.tools` directory beside the repository; substituting `.\php-pgsql.bat` for `php` makes every command below work when PHP is not on PATH. The evaluation scripts find that interpreter themselves, or use the one named in the `PHP` environment variable. The interpreter itself is not committed, because a platform-specific binary does not belong in a source repository, but its version and required extensions are recorded here.

The rest of the toolchain, at the versions used: PostgreSQL 18.2, Python 3.14.0, pandoc 3.9.0.1, and Microsoft Word 16.0, which `tools/paginate_report.py` drives to export the PDFs and to measure the page numbers printed in the contents list. The Python packages the document build imports are pinned in `tools/requirements-docs.txt`. The evaluation scripts use only the Python standard library.

10.2 Commands

The evaluation round, against a local backend:

```
php artisan serve
php artisan tinker tools/seed-evaluation-event.php
python tools/run-evaluation.py
python tools/measure-dashboard-freshness.py
```

The concurrency experiment, which needs PostgreSQL and creates and drops its own scratch database, so no server needs to be running:

```
python tools/run-concurrency-experiment.py
```

These regenerate `tools/evaluation-results.json`, `tools/freshness-results.json` and `tools/concurrency-results.json`. They overwrite the recorded runs, so the figures quoted in this report come from the named runs committed in those files.

Backend test suite. Twelve of the starter kit's page tests need the frontend built first; without it they fail on the missing Vite manifest. The build calls `php artisan`, so PHP must be on PATH while it runs:

```
npm install
npm run build
php artisan test
```

Static analysis:

```
php vendor/bin/phpstan analyse --memory-limit=1G
php vendor/bin/pint --test
```

PHPStan analyses in parallel worker processes that each receive PHP's `memory_limit`, 128 MB by default, which a worker on this project can exhaust; `--memory-limit=1G` makes the run deterministic. Whenever the analysis completes it reports zero errors at level 7.

Mobile:

```
flutter analyze
flutter test
```

Word and PDF deliverables, rebuilt from this markdown and from the report:

```
python tools/build_docx_deliverables.py
python tools/paginate_report.py
```

`tools/paginate_report.py --check` fails if the page numbers printed in the contents lists no longer match the rendered documents.

11. End-to-end voucher trial on a device

Until 14 October 2026 this flow failed at its first step. The attendee app showed only the check-in and session pass (`ATTENDEE-DEMO-001`), and the meal scanner answered that code with 404 "Invalid meal voucher QR token.", because it is not a meal voucher. `Attendee.fromJson` ignored the `meal_vouchers` that `GET /events/{event}/attendees/me` already returned, and no screen showed them. Both are fixed: the model keeps the vouchers, and a My meals tab shows each one with its category and status, and an unused one as a QR code of its `qr_token` (`mobile/lib/features/attendee/presentation/my_meals_screen.dart`).

Setup. One Android emulator, AVD `conference_check` (Pixel 7 profile, Android API 36, emulator 36.6.11), running a debug build of the app at the validated state. The backend was the local Laravel server on PostgreSQL 18.2, reached from the emulator at `10.0.2.2:8000`. Before the trial the demo seed was rerun (`php artisan db:seed`) to move the meal windows to the trial date. The seeded attendee's Breakfast and Lunch vouchers are seeded as redeemed, and Supper, `MEAL-DEMO-1-3`, as unused.

How the code was read. By manual entry. Both roles ran in turn on the one emulator, signed in first as `attendee@example.com` and then as `scanner@example.com`, and an emulator's camera cannot see another screen. The QR code the attendee app showed was decoded off the device from the screenshot, with the `zxing-cpp` library, and reads `MEAL-DEMO-1-3`. Camera reading of the code by a second phone was not done in this trial.

Steps and responses, 14 October 2026 (times UTC):

Step	Time	Action	Response

Step	Time	Action	Response
1	18:07	Attendee app, My meals tab	Supper shown Unused with a QR code; Lunch and Breakfast shown Redeemed with their times (report Figure 7)
2	18:09:25	Scanner app, Meal scanner, `MEAL-DEMO-1-3` entered and submitted	200, "Voucher redeemed successfully." (report Figure 9); voucher `redeemed`, `redeemed_at` 18:09:25
3	18:10:04	Same token submitted again	409; the screen was closed before it was captured
4	18:11:18	Same token submitted a third time	409, "Meal voucher has already been redeemed or is not usable." (report Figure 10)
5	18:12	Same token sent to the API directly, as the scanner	409 with `status: redeemed`, `redeemed_at: 2026-10-14T18:09:25Z`
6	18:12	`ATTENDEE-DEMO-001` sent to the meal scan endpoint	404 "Invalid meal voucher QR token."

The 200 in step 2 and the 409s in steps 3 and 4 are read from the app's messages, which it shows only for those answers, and from the server's request log, which records the three requests at those times; step 5 shows the status code and body directly. **Result: pass**, on one emulator with manual entry. What it does not show is a camera reading the code, or two physical devices.

12. Test cases

Every test case in one table: the 31 component API tests (TC-01 to TC-31), the evaluation steps (TC-32 to TC-46), the Flutter tests that bear on the voucher flow and offline replay (TC-47 to TC-51), and the device trial (TC-52 to TC-55). API and Flutter results are from the runs on 14 October 2026; evaluation results are from the committed results files. The report's Chapter 6 and the slides use this table.

ID	What is tested	Expected result	Actual result	Pass/fail
TC-01	login returns a token	200 with a Sanctum token	200 with token	Pass
TC-02	registration cannot choose the organiser role	account created as attendee	created as attendee	Pass
TC-03	analytics summary	200 with the summary totals	200, totals as seeded	Pass
TC-04	valid meal voucher	200; voucher	as expected	Pass

ID	What is tested	Expected result	Actual result	Pass/fail
	scan	redeemed; one redemption row		
TC-05	second scan of a redeemed voucher	409 with the earlier redemption time	409 with redeemed_at	Pass
TC-06	duplicate caught by the unique constraint	409, not 500	409	Pass
TC-07	unknown meal token	404 Invalid meal voucher QR token	404	Pass
TC-08	redeeming one category leaves the others redeemable	other categories still 200	200 for each other category	Pass
TC-09	valid session attendance scan	200; attendance recorded	as expected	Pass
TC-10	duplicate session scan	409 duplicate	409	Pass
TC-11	check-in that loses a race	duplicate answer, not 500	duplicate, no 500	Pass
TC-12	session scan that loses a race	409, not 500	409	Pass
TC-13	notification send in demo mode	recorded as demo, never sent	demo	Pass
TC-14	missing Firebase config with demo mode off	send fails; nothing marked sent	failed	Pass
TC-15	custom notification target	rejected with 422	422	Pass
TC-16	CSV report download	200, text/csv attachment	as expected	Pass
TC-17	every export has one row per database record	row counts equal the database	equal for all four	Pass
TC-18	pushes go out after recipient rows commit	no push inside the transaction	as expected	Pass
TC-19	send through FCM v1 when	v1 request sent with OAuth token	as expected	Pass

ID	What is tested	Expected result	Actual result	Pass/fail
	configured			
TC-20	delivery recorded per recipient	sent / failed / pending per recipient	as expected	Pass
TC-21	session scan records the scanning device	device id stored	stored	Pass
TC-22	capacity threshold transitions alert organisers	warning and over-capacity pushes to organisers	both sent	Pass
TC-23	attendee fetches own QR token and vouchers	200 with own token and meal_vouchers	as expected	Pass
TC-24	attendee on staff or organiser routes	403	403	Pass
TC-25	attendee opens another attendee's voucher	403	403	Pass
TC-26	notification read by a non-recipient	403; list holds only own	403	Pass
TC-27	device token deleted by a non-owner	403	403	Pass
TC-28	linking an attendee record to a scanner	scanner keeps scanner role	role kept	Pass
TC-29	edit or delete a redemption, or cascade it away	refused; 409 on parent delete	refused / 409	Pass
TC-30	scanner downloads a report	403	403	Pass
TC-31	attendees/me with no linked record	404	404	Pass
TC-32	200 first scans of fresh vouchers	200 of 200 accepted	200 of 200	Pass

ID	What is tested	Expected result	Actual result	Pass/fail
TC-33	100 sequential re-scans of a redeemed voucher	100 of 100 refused (409)	100 of 100	Pass
TC-34	20 cross-category attendees, three categories each	60 of 60 accepted; repeat refused	20 of 20 attendees	Pass
TC-35	20 sequential duplicate session scans	20 of 20 refused	20 of 20	Pass
TC-36	concurrency, lock and constraint on (as built), 50 races	one success per race	50 of 50	Pass
TC-37	concurrency, lock off, constraint on	one success per race	50 of 50	Pass
TC-38	concurrency, lock on, constraint off	one success per race	50 of 50	Pass
TC-39	concurrency negative control, both off	double redemption detected (expected to fail)	48 of 50 redeemed more than once	Pass (control failed as designed)
TC-40	voucher scan latency, n=200	median < 500 ms, p95 < 1 s	145.3 ms / 261.0 ms (loopback)	Pass
TC-41	session scan latency, n=40	median < 500 ms, p95 < 1 s	128.4 ms / 228.7 ms (loopback)	Pass
TC-42	dashboard freshness, 20 injected check-ins (poller, not the app)	each visible within 30 s	20 of 20 in recorded run; 19 of 20 in supervisor's rerun	Pass in recorded run; not guaranteed
TC-43	attendance.csv against SQL	rows and checked-in total equal	500 / 40, equal	Pass
TC-44	meals.csv against SQL	rows and per-category totals equal	280 / 94, 93, 93, equal	Pass
TC-45	sessions.csv against SQL	rows and per-session totals equal	10 rows; one non-zero session, equal	Pass
TC-46	notifications.csv against SQL	rows and recipients equal	0 against 0 (unit test is the real check)	Pass
TC-47	My meals parses	vouchers with	as expected	Pass

ID	What is tested	Expected result	Actual result	Pass/fail
	vouchers from attendees/me	category and status kept		
TC-48	My meals shows a QR only for an unused voucher	one QR, for the unused voucher	one QR	Pass
TC-49	replay settles only on 404, 409 and 422	other answers keep the scan	as expected	Pass
TC-50	queued scans survive a restart	replayed on the next run	as expected	Pass
TC-51	manual token trimming	surrounding spaces removed	as expected	Pass
TC-52	attendee app shows the meal voucher	Supper voucher shown as QR, status Unused	shown; QR decodes to MEAL-DEMO-1-3	Pass
TC-53	first scan of MEAL-DEMO-1-3 from the scanner app	200, Voucher redeemed successfully	200; redeemed 18:09:25 UTC	Pass
TC-54	second scan of the same token	409, already redeemed	409, twice	Pass
TC-55	attendee pass token at the meal scanner	404 Invalid meal voucher QR token	404	Pass

TC-42 measures a poller that reproduces the app's refresh, not the app on a device (Section 4). TC-46 compares zero with zero and is backed by TC-17. TC-40 and TC-41 were measured over loopback on one machine (Section 3). Not in the table because they were not run or not measured: the manual baseline, push delivery across a fleet of devices, and a timed capacity alert.